

Juego con obstáculos en ESP32 y OLED

Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio
Electrónica Digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	4
5. Estructura del código	4
6. Explicación de funciones principales	5
7. Comunicación	5
8. Interfaz de usuario	5
9. Procedimiento de prueba	5
10. Manejo de errores y seguridad	6

1. Identificación del proyecto

- **Nombre del proyecto:** Juego con obstáculos en ESP32 y OLED
- **Integrantes del equipo:** Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio

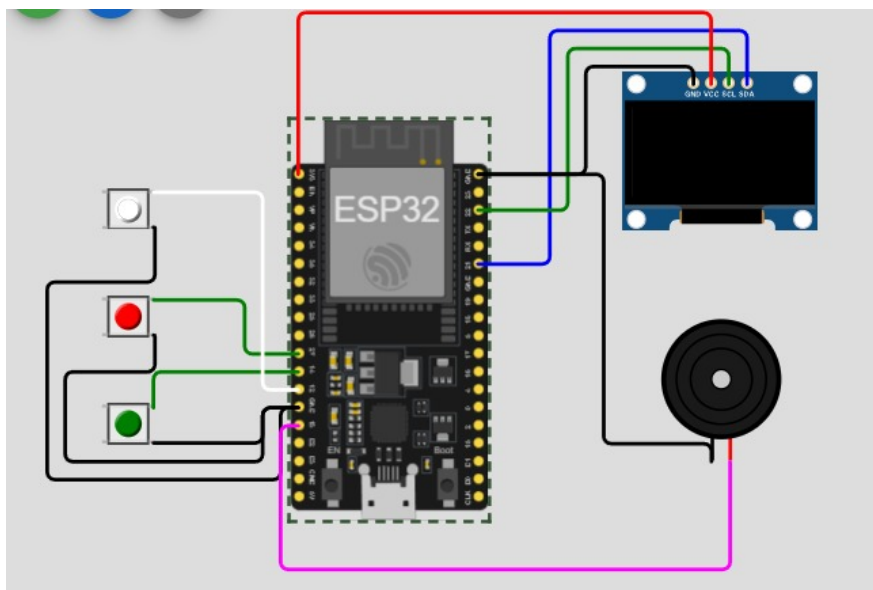
2. Resumen

En esta práctica se diseñó e implementó un videojuego interactivo utilizando el microcontrolador ESP32, programado en MicroPython. El sistema permite al usuario controlar un personaje en una pantalla OLED SSD1306, con el objetivo de esquivar obstáculos en movimiento y acumular puntaje. El juego cuenta con tres modos de dificultad: clásico, contra-tiempo y hardcore, cada uno con diferentes parámetros de velocidad y generación de obstáculos. Se emplearon botones físicos para el control del jugador y un buzzer para la retroalimentación sonora. El programa implementa una máquina de estados para gestionar las diferentes etapas del juego (menú, ejecución, pausa y fin), así como funciones para animación, detección de colisiones y actualización dinámica de la dificultad. Se utilizaron temporizadores basados en ticks para el manejo del tiempo sin bloquear la ejecución. Este proyecto permitió afianzar conceptos como manejo de periféricos (GPIO, PWM, I2C), estructuras de control, programación en tiempo real y desarrollo de interfaces gráficas básicas en sistemas embebidos.

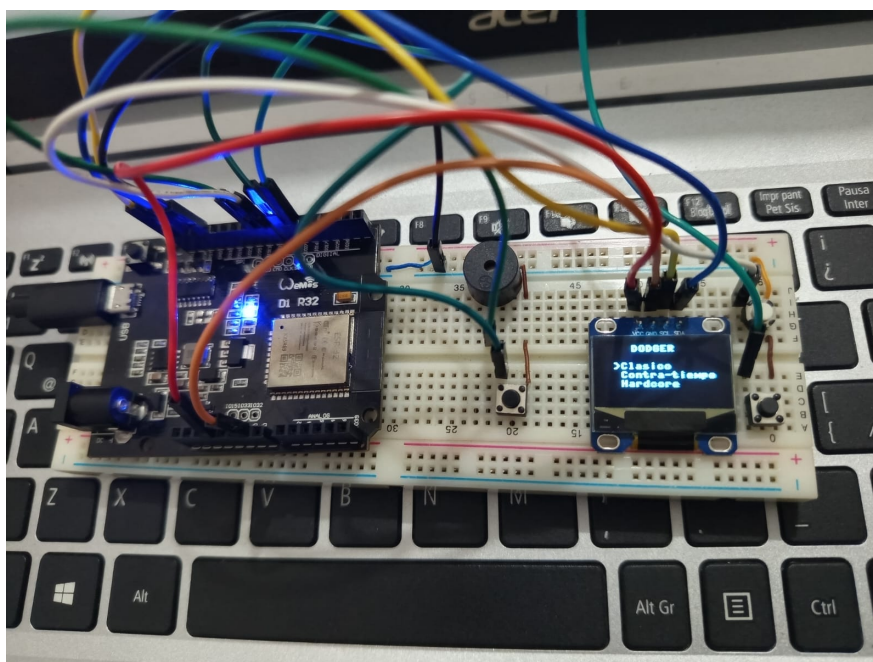
3. Descripción del hardware

- **Microcontrolador:** ESP32 encargado de ejecutar la lógica del videojuego.
- **Entradas:**
 - **Botón UP (pin 14):** mueve el personaje hacia arriba
 - **Botón DOWN (pin 27):** mueve el personaje hacia abajo
 - **Botón START (pin 12):** iniciar/pausar juego
- **Salidas:**
 - **Pantalla OLED SSD1306:** visualización del juego
 - **Buzzer (pin 13):** genera sonidos de interacción
- **Fuente de alimentación:** 5V por USB

Diagrama de conexión:



Montaje físico:



4. Descripción del software

- **Lenguaje:** MicroPython
- **Librerías:** machine, time, random, ssd1306
- **IDE:** Thonny

Flujo general:

1. Configuración de pines (botones, buzzer, I2C)
2. Inicialización de pantalla OLED
3. Definición de modos de juego y sprites
4. Inicialización de variables globales
5. Ejecución del ciclo principal (while True)
6. Lectura de entradas (botones)
7. Actualización del estado del juego
8. Detección de colisiones
9. Representación visual en tiempo real de los elementos del videojuego.
10. Generación de sonidos con buzzer

5. Estructura del código

- Archivo principal: `main.py`
- Variables globales: estado del juego (MENU, JUEGO, PAUSA, GAME OVER), Puntaje, obstáculos, velocidad, modo.
- Funciones:
 - `dibujar_sprite()` dibuja gráficos en la OLED
 - `beep()` genera sonido con PWM
 - `reset()` reinicia el juego
 - `animacion()` alterna sprites del jugador
 - `actualizar()` mueve obstáculos y aumenta dificultad
 - `colision()` detecta colisiones
 - `dibujar()` Dibujo del personaje, obstáculos y puntaje en la pantalla OLED.

6. Explicación de funciones principales

`dibujar_sprite()` : *Dibujar gráficos pixel a pixel en la pantalla OLED usando matrices binarias.*

`beep()`: Genera sonido mediante PWM en el buzzer con frecuencia y duración definidas.

`reset()`: Inicializa variables del juego como puntaje, posición del jugador y obstáculos.

`animacion()`: Alterna entre dos sprites para simular movimiento del personaje.

`actualizar()`: Controla la aparición de obstáculos, su movimiento y el incremento del puntaje. También ajusta la dificultad.

`colision()`: Evalúa si el jugador entra en contacto con un obstáculo mediante comparación de posiciones.

`dibujar()`: Actualiza la pantalla dependiendo del estado del juego (menú, juego, pausa o fin).

7. Comunicación

La comunicación en el sistema se realiza mediante:

I2C: comunicación entre el ESP32 y la pantalla OLED

Señales digitales: lectura de botones

PWM: generación de sonido en el buzzer

8. Interfaz de usuario

Pantalla OLED: muestra menú, puntaje, modo de juego y animaciones

Botones físicos: control del jugador y navegación del menú

Buzzer: retroalimentación auditiva

Elementos visuales:

Personaje animado (conejo)

Obstáculos

Puntaje y estadísticas

Indicadores de estado (PAUSA, GAME OVER)

9. Procedimiento de prueba

1. Conectar el ESP32 al computador mediante USB.
2. Abrir el IDE Thonny.
3. Cargar el archivo main.py.

4. Ejecutar el programa.
5. Seleccionar el modo de juego con los botones.
6. Iniciar el juego con el botón START.
7. Verificar el movimiento del personaje.
8. Observar el comportamiento de los obstáculos.
9. Confirmar la detección de colisiones.
10. Validar el sonido del buzzer.
11. Probar la pausa y reinicio del juego.

10. Manejo de errores y seguridad

Uso de límites en la posición del jugador para evitar salidas de pantalla.

Control de tiempo con `time.sleepms()` *para evitar rebotes en botones*.

Manejo de listas para evitar desbordamientos de memoria.

Reinicio controlado del juego con la función `reset()`.

Uso de condiciones para evitar valores negativos en temporizadores.